MyCodeSchool

exercițiu 06 — pipelines

„Wait for 2 hours” iese din test și intră în GitHub Actions · Playwright / TypeScript · Andreea

Unde lucrezi: `sap-starter/` și rădăcina repo-ului (`playwright-oop/`), peste rezolvarea ta de la exercițiul 05.
Nou, ai voie: `webServer` în config, tag-uri pe teste, `node:fs` , `toPass` , fișiere `.yml` în `.github/workflows/` .
Înainte să începi: `npx tsc --noEmit` fără erori și `12 passed` în `tests/contract-upsell` .
Teorie: fișa `teorie-contract-pipeline.pdf` , secțiunile 6–8.

Ideea

Etapa	Pașii din fișa ta	Tag	Când rulează
act	1–6: contract, upsell, Ent	`@act`	la fiecare push
verify	7–8: Schedule actions	`@after-batch`	imediat după act *și* programat, noaptea

Partea 1 — local

1. Suita pornește singură aplicația

nivel 1

În `playwright.config.ts` , în locul blocului `webServer` comentat:

```
webServer: {
  command: 'npm run dev -- --host 127.0.0.1 --port 5173 --strictPort',
  cwd: '../sap-ui',
  url: 'http://127.0.0.1:5173',
  reuseExistingServer: !process.env.CI,
},
```

Gata când: închizi terminalul în care rula `sap-ui` , rulezi `npx playwright test tests/contract-upsell --project=chromium` și ai tot `12 passed` .

De reținut: în pipeline nu e nimeni care să pornească aplicația de mână.

2. Cazurile, într-un fișier comun

nivel 1

Două fișiere de test vor folosi aceleași cazuri. Mută tipul `ContractCase` , constantele ajutătoare și lista `contractCases` din spec în `utils/contract-cases.ts` , cu `export` . În spec rămâne doar importul.

Gata când: tsc curat și `12 passed` .

3. Două etape, două tag-uri

mai greu

1. În `contract-upsell.spec.ts` , fiecare `describe` primește tag-ul: `test.describe("BEGINNER - ...", { tag: "@act" }, () => { ... })` .
2. Testul „scheduled actions” *iese* din acest fișier. Blocul EXPERIENCED rămâne doar cu regula fazelor.
3. În `package.json` , la `scripts` :

```
"test:act":
  "playwright test --project=chromium --grep-invert @after-batch",
"test:verify":
  "playwright test --project=chromium --grep @after-batch",
```

Gata când: `npx playwright test tests/contract-upsell --project=chromium` dă `9 passed` .

4. Predarea ID-urilor

mai greu

Fișier nou `utils/run-data.ts`:

```
import fs from "node:fs";
import path from "node:path";

export type RunData = { contractId: string; entId: string; docNo: string };

const runDataDir = path.resolve("run-data");

export function saveRunData(data: RunData) {
  fs.mkdirSync(runDataDir, { recursive: true });
  const file = path.join(runDataDir, `${data.contractId}.json`);
  fs.writeFileSync(file, JSON.stringify(data, null, 2));
}

export function loadRunData(): RunData[] {
  if (!fs.existsSync(runDataDir)) {
    return [];
  }
  return fs.readdirSync(runDataDir).sort().map((fileName) =>
    JSON.parse(fs.readFileSync(path.join(runDataDir, fileName), "utf-8")));
}
```

1. În testul BEGINNER, după `toEqual`: ia ultimul document citit și cheamă `saveRunData({ contractId: c.contractId, entId: ..., docNo: ... })`.
2. În `.gitignore` din rădăcina repo-ului: `run-data/`.

Gata când: după `npm run test:act` există `run-data/C-0100.json`, `C-0200.json`, `C-0300.json`, fiecare cu cele 3 ID-uri.

5. Testul de după job

mai greu

Fișier nou `tests/contract-upsell/scheduled-actions.after-batch.spec.ts`. Sus: `const runData = loadRunData();`. Un `describe` cu `{ tag: "@after-batch" }`:

1. Un test care verifică predarea: lista de `contractId` din `runData` este egală cu cea din `contractCases`.
2. O buclă peste `runData`. Cazul îl găsești cu `contractCases.find(...)` după `contractId`, iar documentul îl iei din `data.docNo`, nu din caz.
3. Verificarea reîncearcă până trece sau expiră timpul:

```
await expect(async () => {
  await scheduledActionsPage.searchDocument(data.docNo);
  await expect(scheduledActionsPage.resultCount)
    .toHaveText(`${c!.expected.actions.length} rows`);
  expect(await scheduledActionsPage.readRows())
    .toEqual(c!.expected.actions);
}).toPass({ timeout: 20_000, intervals: [2_000] });
```

Rezultat așteptat: `npm run test:verify` → `4 passed`.

Gata când: ștergi folderul `run-data` și `npm run test:verify` pică la primul test; rulezi `npm run test:act`, apoi `npm run test:verify` dă `4 passed`.

De reținut: verify nu știe nimic singur. Tot ce caută a primit de la act, prin fișier.

Partea 2 — GitHub Actions

6. Primul workflow: act, apoi verify

mai greu

Fișier nou, în rădăcina repo-ului: `.github/workflows/tests.yml`.

```
name: tests
on:
  push:
    branches: [main]
  pull_request:
defaults:
  run:
    working-directory: sap-starter
jobs:
  act:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm ci
        working-directory: sap-ui
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx tsc --noEmit
      - run: npm run test:act
      - uses: actions/upload-artifact@v4
        with:
          name: run-data
          path: sap-starter/run-data
          if-no-files-found: error
```

Al doilea job, `verify`, îl scrii tu, sub `act`:

1. `needs: act`, apoi aceiași pași până la `playwright install` inclusiv.
2. În loc de upload: `actions/download-artifact@v4` cu `name: run-data` și `path: sap-starter/run-data`.
3. Ultimul pas: `npm run test:verify`.

Gata când: după push, în tab-ul Actions din GitHub ambele joburi sunt verzi, iar rularea are artefactul `run-data`.

7. Al doilea workflow: verify programat și la cerere

mai greu

Fișier nou `.github/workflows/verify-after-batch.yml`. Jobul `verify` e același, cu două diferențe:

```
on:
  schedule:
    - cron: "0 3 * * *"
  workflow_dispatch:
permissions:
  actions: read
  contents: read
```

```
      - name: Download run-data from the latest green tests run
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          run_id=$(gh run list --workflow tests.yml --branch main \
            --status success --limit 1 \
            --json databaseId --jq '.[0].databaseId')
          gh run download "$run_id" --name run-data --dir run-data
```

Gata când: în Actions alegi `verify-after-batch` → Run workflow și rularea e verde.

De reținut: `schedule` = jobul fix, de noapte. `workflow_dispatch` = jobul pornit la cerere.

8. Dovada

mai greu

1. Local, în `utils/contract-cases.ts`, la acțiunea `OPEN_PHASE` a upsell-ului schimbă `"Done"` în `"Planned"`. Rulează `npm run test:verify`. Pune la loc.

Rezultat așteptat: testele pentru `C-0200` și `C-0300` pică după 20 de secunde, cu diff-ul pe `status`.

Întrebări de răspuns în scris:

1. De ce nu punem `waitForTimeout` de 2 ore în test?
2. Ce prinde rularea programată și nu prinde rularea de la push?

Gata când: ai văzut diff-ul, totul e la loc (`4 passed`) și ai răspuns la întrebări.